

End Sem Exam — CS0.101 Computer Programming — Monsoon '25 Semester

Questions compiled from given answer rubric

Q1. (3 marks) Write a function to check whether a number is prime or not.

Q2. (3 marks) Given an array of positive integers `arr[]` of size `n`, write a function to find the **second largest** element in the array.

Q3. (3 marks) Assuming an 8-bit representation, express -32 in **2's complement** notation. Show all working steps.

Q4. (3 marks) Consider the following function implemented in C. What is the output of `printxy(1, 1)`?

```
void printxy(int x, int y) {
    int *ptr;
    x = 0;
    ptr = &x;
    y = *ptr;
    *ptr = 1;
    printf("%d,%d", x, y);
}
```

Q5. (3 marks) What will this code print?

```
#include <stdio.h>
void mystery(int *ptrb, int *ptrb) {
    int *temp;
    temp = ptrb;
    ptrb = ptrb;
    ptrb = temp;
}
int main() {
    int a=2025, b=0, c=4, d=42;
    mystery(&a, &b);
    if (a < c) mystery(&c, &a);
    mystery(&a, &d);
    printf("%d", a);
}
```

Q6. (3 marks) What will this code print?

```
#include <stdio.h>
int main() {
    int x = 1, z[2] = {10, 11};
    int *p = NULL;
    p = &x;
    *p = 10;
    p = &z[1];
    *(&z[0] + 1) += 3;
    printf("%d, %d, %d\n", x, z[0], z[1]);
    return 0;
}
```

Q7. (3 marks) Given a singly linked list, fill in the code of the recursive function `sumList()` to return the sum of all node values.

```
typedef struct {
    int data;
    struct Node* next;
} Node;

int sumList(Node* head) {
    /* Fill in the code here */
}
```

Q8. (3 marks) Fill in the blanks to complete the C program that dynamically allocates an array of size `n`, reads `n` integers, and prints their sum.

```
int n;
scanf("%d", &n);
int *a = _____; // Blank 1
for(int i=0; i<n; i++) scanf("%d", &a[i]);
int sum = 0;
for(int i=0; i<n; i++) sum += _____; // Blank 2
printf("%d", sum);
```

Q9. (5 marks) You are given strings A and B . You can: (i) delete any character from A with cost x , or (ii) delete any character from B with cost y . Find the minimum cost to make both strings equal by deletions only. Give **only** the relevant dynamic programming recurrence relation, clearly explaining all symbols used.

Q10. (3 marks) What is the output of the following program when we call `fun(3)`?

```
void fun(int n) {
    if (n == 0)
        return;
    fun(n - 1);
    printf("%d ", n);
    fun(n - 1);
}
```

Q11. (3 marks) Write a program that fills an $n \times n$ square matrix as follows: the **right-to-left diagonal** with 0s, the **lower-right triangle** with -1 s, and the **upper-left triangle** with $+1$ s.

Q12. (3 marks) What is the output of the following program?

```
int main(void) {
    int a[] = {10, 20, 30};
    int b[] = {40, 50, 60, 70};
    int *pr[] = {a, b+1, a+2};
    int **pp = pr;
    printf("%d %d %d", pr[0][1], pr[1][1], pr[2][0]);
    printf("\n");
    printf("%d %d\n", pp[1][0], *(pp[2]));
    return 0;
}
```

Q13. (4 marks) What is the output of the following program?

```
int fun(int count) {
    if(count < 3) {
        ++count;
        fun(fun(count));
    }
    printf("%d ", count);
    return count;
}
int main() {
    fun(1);
    return 0;
}
```

Q14. (3 marks) Write a program (only the `main` function contents) that approximates the value of e using a loop that terminates when the difference between two successive approximations is less than 0.00001. Use the series: $e = 1 + \frac{1}{1!} + \frac{1}{2!} + \frac{1}{3!} + \dots$

Q15. (5 marks) Consider a C program to store student exam records. Each student has `roll` (int), `name` (char*), and `marks[3]` (marks in 3 subjects).

- (a) (1 mark) Define the structure `Student`.
- (b) (1 mark) Declare an array of 8 students in `main`.
- (c) (2 marks) Write a function that receives the array and a subject index (0–2), and returns the highest mark in that subject.
- (d) (1 mark) In `main`, call the function for subjects 0, 1, and 2 and print the highest mark for each.

Q16. (5 marks) Given the definitions below, complete `mergeSorted()` which merges two sorted singly linked lists into a single non-decreasing sorted list. Fill in `TODO 1` through `TODO 9`.

```
typedef struct Node { int data; struct Node *next; } Node;

Node* mergeSorted(Node *a, Node *b) {
    if (a == NULL) return /* TODO-1 */;
    if (b == NULL) return /* TODO-2 */;

    Node *head = NULL, *tail = NULL;

    if (/* TODO-3 */) {
        head = tail = a; a = a->next;
    } else {
        head = tail = b; b = b->next;
    }

    while (/* TODO-4 */) {
        if (/* TODO-5 */) {
            tail->next = a; a = /* TODO-6 */;
        } else {
            tail->next = b; b = /* TODO-7 */;
        }
    }
}
```

```

    }
    tail = tail->next;
}

if (a != NULL) tail->next = /* TODO-8 */;
if (b != NULL) tail->next = /* TODO-9 */;
return head;
}

```

Q17. (5 marks) The following recursive Fibonacci program is inefficient due to repeated computation. Modify it to use **memoization** so each value is computed only once.

```

long int fib(int n) {
    if (n == 0) return 0;
    if (n == 1) return 1;
    return fib(n - 1) + fib(n - 2);
}

```

Q18. (3 marks) What will be the output?

```

int arr[5] = {1, 2, 3, 4, 5};
int *p = arr + 1;
int *q = arr + 3;
printf("%d", q - p);

```

Q19. (3 marks) What will be the output?

```

int *p = (int*)malloc(5 * sizeof(int));
for(int i = 0; i < 5; i++)
    *(p + i) = i * 2;
int **pp = &p;
printf("%d", (*pp)[3]);
free(p);

```

Q20. (3 marks) What will be the output?

```

char *str = "Programming";
char *ptr = str;
while(*ptr != 'g') ptr++;
printf("%c", *(ptr + 2));

```

Q21. (3 marks) What will be the output if "input.txt" initially contains "123 456 789"?

```

FILE *fp = fopen("input.txt", "r+");
int a, b;
fscanf(fp, "%d %d", &a, &b);
fprintf(fp, "%d", a + b);
rewind(fp);
int x, y, z;
fscanf(fp, "%d %d %d", &x, &y, &z);
printf("%d %d %d", x, y, z);
fclose(fp);

```

Q22. (3 marks) What is the value printed by the following code?

```
enum Color { RED = 5, GREEN, BLUE = 10, YELLOW };
printf("%d", YELLOW);
```

Q23. (3 marks) What will be the output?

```
int x = 100;
int *p = &x;
int **pp = &p;
int ***ppp = &pp;
***ppp = 200;
printf("%d", x);
```

Q24. (3 marks) What will be the output?

```
int arr[3][3] = {{1,2,3},{4,5,6},{7,8,9}};
int (*p)[3] = arr;
printf("%d", *((p+1)+2));
```

Q25. (3 marks) What will be the output?

```
struct Node { int data; char flag; };
struct Node nodes[3] = {{10, 'A'}, {20, 'B'}, {30, 'C'}};
struct Node *ptr = nodes;
ptr++;
printf("%d%c", (ptr-1)->data, (ptr+1)->flag);
```

Q26. (3 marks) What will be printed?

```
char *arr[] = {"Hello", "World", "Test"};
char **ptr = arr;
ptr++;
printf("%c%c", (*ptr)[1], (*(ptr-1))[4]);
```

Q27. (3 marks) What will be the output?

```
struct Data { int value; struct Data *next; };
struct Data d1 = {10, NULL};
struct Data d2 = {20, &d1};
struct Data d3 = {30, &d2};
d1.next = &d3;
printf("%d", d3.next->next->next->value);
```